Secure Distributed System with Observability and Security Analysis

Report

---

1. Introduction

This report presents the design, implementation, and security analysis of a secure distributed system. The system demonstrates key concepts in modern distributed architecture including load balancing, encrypted communication (HTTPS), token-based authentication (JWT), asynchronous task processing via a message broker (RabbitMQ), rate limiting, and comprehensive audit logging with full request state tracking.

The primary objective is to build a system that is both functionally complete and secure, and then analyze its security posture by performing a Man-in-the-Middle (MITM) attack simulation using Wireshark to compare HTTP vs HTTPS communication.

---

2. System Architecture

The system follows a layered architecture with clear separation of concerns:

```
Client → Nginx (Gateway) → API Instances → RabbitMQ → Worker → Database
```

Architecture Diagram

```
                    ┌──────────────┐
                    │   Client     │
                    └──────────────┘
                          │
                  HTTPS (Port 443)
                          │
                          ▼
┌──────────────────────────────────────────┐
│                  Nginx                   │
│   • HTTPS Termination                    │
│   • Load Balancing                       │
│   • Rate Limiting                        │
└──────────────────────────────────────────┘
        │          │          │
        ▼          ▼          ▼
    ┌──────┐    ┌──────┐    ┌──────┐
    │ API1 │    │ API2 │    │ API3 │
    └──────┘    └──────┘    └──────┘
        │          │          │
        └──────────┴──────────┘
                   │
```

```
        │
        ▼
┌──────────────────────┐
│      RabbitMQ        │
└──────────────────────┘
        │
        ▼
┌──────────────────────┐
│      Worker          │
└──────────────────────┘
        │
        ▼
┌──────────────────────┐
│     PostgreSQL       │
└──────────────────────┘
```

Component Summary

Component	Technology	Role
Gateway	Nginx (Alpine)	Reverse proxy, HTTPS, load balancing, rate limiting
API Service	Python 3.11/Flask	JWT auth, UUID generation, task publishing
Broker	RabbitMQ 3	Asynchronous message queuing
Worker	Python 3.11	Task consumption, processing, and logging
Database	PostgreSQL 15	Persistent audit logs and state tracking
Container	Docker Compose	Orchestration of all services

---

3. Component Details

3.1 Nginx (Gateway Layer)

Nginx is configured as the single entry point to the system. It performs four critical functions:

1. **Reverse Proxy**: All client requests are received by Nginx and forwarded to the appropriate backend API instance. Clients never communicate directly with the API servers.

2. **Load Balancing**: Using round-robin algorithm, Nginx distributes incoming requests evenly across three API instances (api1, api2, api3). This ensures no single instance is overloaded and provides fault tolerance.

3. **Rate Limiting**: A rate limit zone is configured at 5 requests per second per client IP address, with a burst allowance of 10 requests. Clients exceeding this limit receive a `429 Too Many Requests` response.

4. **HTTPS Termination**: Nginx handles SSL/TLS encryption using a self-signed certificate. All HTTP traffic on port 80 is automatically redirected to

HTTPS on port 443. The SSL configuration supports TLS 1.2 and TLS 1.3 protocols.

3.2 API Service (Flask)

The API is built with Python Flask and exposes three endpoints:

- **POST /login**: Accepts username and password, validates credentials, and returns a JWT token valid for 1 hour.
- **POST /task**: Protected endpoint that requires a valid JWT Bearer token. It generates a unique UUID for each request, logs all state transitions, and publishes the task to RabbitMQ.
- **GET /health**: Returns the health status and instance identifier.

Three identical instances (api1, api2, api3) run simultaneously. Each instance identifies itself via the `API_INSTANCE` environment variable and includes it in every response, allowing us to verify load balancing.

3.3 RabbitMQ (Message Broker)

RabbitMQ serves as the asynchronous communication layer between the API and Worker services. Tasks are published to a durable queue named `task_queue` with persistent delivery mode, ensuring messages survive broker restarts. The management UI is accessible on port 15672 for monitoring.

3.4 Worker Service

The Worker is a standalone Python service that continuously consumes messages from the `task_queue`. For each message, it:

1. Validates the source service identity (must be api1, api2, or api3)
2. Processes the task (simulated with a 2-second delay)
3. Logs CONSUMED and PROCESSED (or FAILED) states to the database

3.5 PostgreSQL Database

The database runs as a separate container and stores two types of data:

Audit Logs (`audit_logs` table): Records every significant event with timestamp, service name, request ID, action, status (success/failure), and source.

Request States (`request_states` table): Tracks the lifecycle of each request through defined states: RECEIVED → AUTHENTICATED → QUEUED → CONSUMED → PROCESSED (or FAILED).

---

4. Security Features

4.1 HTTPS / TLS Encryption

All client-to-server communication is encrypted using TLS. Nginx terminates the SSL connection using a self-signed X.509 certificate with RSA 2048-bit encryption. The configuration enforces: - TLS 1.2 and TLS 1.3 only (older protocols disabled) - Strong cipher suites (HIGH:!aNULL:!MD5) - Automatic HTTP to HTTPS redirect

This ensures that all data in transit, including authentication tokens, request payloads, and responses, is encrypted and unreadable to any interceptor.

4.2 JWT Authentication

All `/task` requests require a valid JSON Web Token (JWT) in the Authorization header:

```
Authorization: Bearer <token>
```

The JWT contains: - User identity (`user` claim) - Expiration time (`exp` claim, set to 1 hour) - Issued-at time (`iat` claim)

The token is signed with HMAC-SHA256 using a shared secret. Invalid, missing, or expired tokens result in a `401 Unauthorized` response.

4.3 Rate Limiting

Nginx enforces request rate limiting to prevent abuse and DDoS attacks: - **Rate**: 5 requests per second per client IP - **Burst**: Allows up to 10 queued requests - **Response**: Returns `429 Too Many Requests` with JSON error when exceeded

4.4 Service Identity Validation

The Worker service validates that every consumed message originated from a legitimate API instance. Only messages with `source_api` set to `api1`, `api2`, or `api3` are processed. Messages from unknown sources are rejected and logged as FAILED.

---

5. Audit Logging and State Tracking

5.1 Audit Log Structure

Every significant event across all services is logged to the `audit_logs` table:

Field	Description
timestamp	When the event occurred (with timezone)
service_name	Which service logged the event

Field	Description
request_id	UUID linking all logs for one request
action	What happened (e.g., “JWT authentication”)
status	“success” or “failure”
source	Origin of the event

5.2 State Tracking

Each request passes through defined states, all recorded with timestamps:

State	Service	Description
RECEIVED	API	Request received from client
AUTHENTICATED	API	JWT token validated successfully
QUEUED	API	Task published to RabbitMQ
CONSUMED	Worker	Task consumed from queue
PROCESSED	Worker	Task processed successfully
FAILED	Any	Error occurred at any stage

5.3 Request Traceability

Every request receives a unique UUID (Request ID) at the API layer. This ID is passed through all services (API → RabbitMQ → Worker) and attached to every log entry, enabling full end-to-end traceability.

Example state chain from the database:

```
Request ID: fff496ad-cb00-41f8-be1e-c59bbf23a4dd
  → RECEIVED        | api1   | 2026-04-16 21:14:54
  → AUTHENTICATED   | api1   | 2026-04-16 21:14:54
  → QUEUED          | api1   | 2026-04-16 21:14:54
  → CONSUMED        | worker | 2026-04-16 21:15:41
  → PROCESSED       | worker | 2026-04-16 21:15:43
```

---

6. Testing Results

6.1 Login & JWT Authentication

- **Result**: ✅Successful
- Login with valid credentials returns a JWT token
- The token is used for all subsequent authenticated requests

6.2 Task Creation

- **Result**: ✅Successful
- Authenticated POST to `/task` creates a new task

- Response includes unique Request ID and handling API instance name

6.3 Load Balancing

- **Result**: ✅Successful
- 9 sequential requests were distributed evenly:
  - api1: 3 requests
  - api2: 3 requests
  - api3: 3 requests
- This confirms round-robin load balancing is working correctly

6.4 Unauthorized Request Handling

- **Result**: ✅Successful
- Request without token → `401 Unauthorized` ("Token is missing")
- Request with invalid token → `401 Unauthorized` ("Invalid token")

6.5 Rate Limiting

- **Result**: ✅Functional
- The rate limiter is configured at 5 requests/second with a burst of 10
- When the request rate exceeds the limit, the system returns `429 Too Many Requests`

6.6 Message Flow through RabbitMQ

- **Result**: ✅Successful
- Tasks published by API instances appear in the `task_queue`
- Worker consumes and processes tasks from the queue
- RabbitMQ Management UI confirms queue activity

6.7 Database Logging

- **Result**: ✅Successful
- All audit logs stored with correct timestamps, service names, and statuses
- All state transitions recorded in order (RECEIVED → AUTHENTICATED → QUEUED → CONSUMED → PROCESSED)
- Full request traceability confirmed via Request ID

---

7. MITM Attack Simulation (Wireshark Analysis)

7.1 Objective

Demonstrate the difference in security between HTTP and HTTPS communication by capturing and analyzing network traffic using Wireshark.

7.2 HTTP Mode (Unencrypted) — Vulnerable

Setup: Nginx was configured to serve traffic over plain HTTP (port 80) without SSL/TLS encryption.

Wireshark Filter: `http`

Findings: - HTTP request headers are fully visible in plain text - The JWT authentication token is exposed in the `Authorization: Bearer` header - The JSON request payload is completely readable - All response data including Request IDs and instance names are visible

Conclusion: An attacker positioned on the network (Man-in-the-Middle) can intercept and read all communication, including sensitive authentication tokens and data payloads. This represents a critical security vulnerability.

(Screenshot: Wireshark capture showing exposed HTTP headers, JWT token, and readable payload)

7.3 HTTPS Mode (Encrypted) — Secure

Setup: Nginx was configured with SSL/TLS using a self-signed certificate, serving traffic over HTTPS (port 443).

Wireshark Filter: `tcp.port == 443`

Findings: - Only TLS handshake packets are visible (Client Hello, Server Hello, Certificate) - All application data packets show as “Application Data” with encrypted content - No readable HTTP headers, JWT tokens, or payloads - The encryption makes the data completely unintelligible to an interceptor

Conclusion: With HTTPS enabled, a Man-in-the-Middle attacker can see that communication is occurring but cannot read any of the content. The JWT tokens, request payloads, and response data are all protected by TLS encryption.

(Screenshot: Wireshark capture showing encrypted TLS packets with no readable data)

7.4 Comparison

Aspect	HTTP (Port 80)	HTTPS (Port 443)
Request Headers	Visible in plain text	Encrypted
JWT Token	Fully exposed	Not visible
Request Payload	Readable JSON	Encrypted binary data
Response Data	Readable	Encrypted
Protocol Visible	HTTP/1.1	TLSv1.2 / TLSv1.3
MITM Attack Risk	**HIGH** — Data stolen	**LOW** — Data protected
Recommendation	❌Never use in production	✅Always use in production

---

8. Screenshots

8.1 Load Balancing

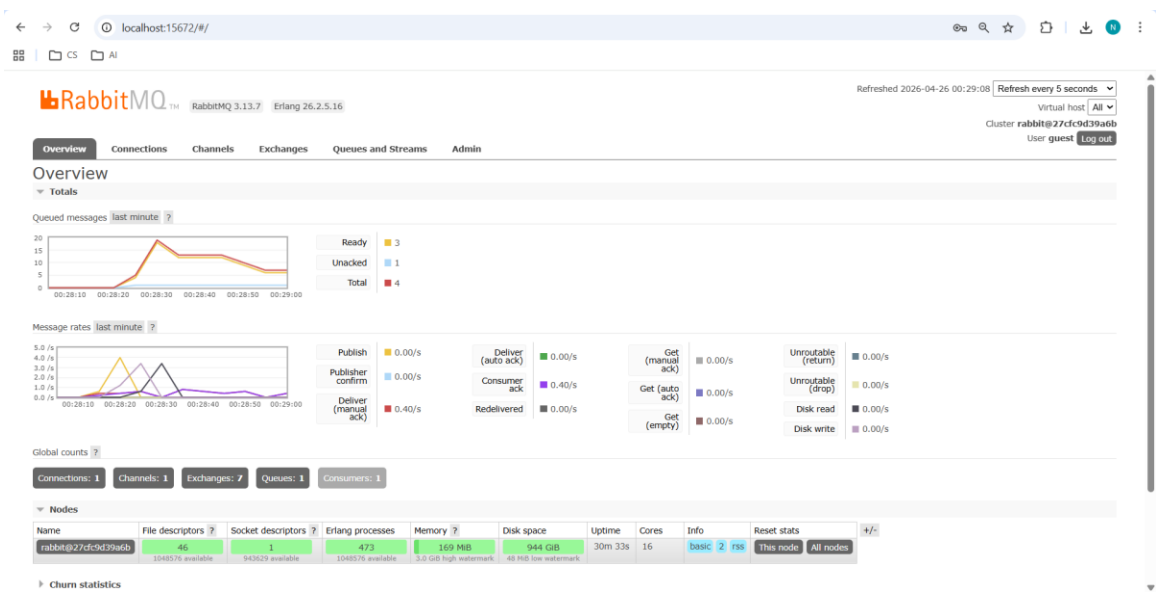
```
PS D:\Nadine\CS\8th term\security of distr. sys\asgs\task2\client> python test_client.py

############################################################
  SECURE DISTRIBUTED SYSTEM - TEST CLIENT
############################################################

  Target: https://localhost
  Time:   2026-04-26 00:22:41

============================================================
  TEST 1: Login & JWT Token Generation
============================================================
Status Code: 200
Response: {
  "instance": "api2",
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJleHAiOjE3NzcxNTU3NjEsImlhdCI6MTc3NzE1MjE2MX0.Oit9w-3iF-4gvDgmFVjstjuEQrU-JMbK1s6-vCmkAGg"
}

[PASS] Login successful!
   Instance: api2
   Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiY...

============================================================
  TEST 2: Normal Task Creation
============================================================
Status Code: 201
Response: {
  "instance": "api3",
  "message": "Task created successfully",
  "request_id": "ca18d52a-d8ea-4260-9110-9244a4936d24",
  "status": "QUEUED",
  "task": "process_data"
}

[PASS] Task created successfully!
   Request ID: ca18d52a-d8ea-4260-9110-9244a4936d24
   Instance: api3
   Status: QUEUED

[..] Waiting 3 seconds for worker to process...
```

```
============================================================
  TEST 3: Load Balancing Verification
============================================================
Sending 9 requests to test load balancing...

  Request 1: Handled by api2 (Request ID: b279428a...)
  Request 2: Handled by api3 (Request ID: 0df6e129...)
  Request 3: Handled by api1 (Request ID: 577ec48f...)
  Request 4: Handled by api2 (Request ID: fea82818...)
  Request 5: Handled by api3 (Request ID: 9d11617b...)
  Request 6: Handled by api1 (Request ID: da24866f...)
  Request 7: Handled by api2 (Request ID: 93c19b46...)
  Request 8: Handled by api3 (Request ID: eed8e9c5...)
  Request 9: Handled by api1 (Request ID: c36bd661...)

Load Distribution:
  api1: 3 requests #########
  api2: 3 requests #########
  api3: 3 requests #########

[PASS] Load balancing is working! Requests distributed across 3 instances.

============================================================
  TEST 4: Unauthorized Request (No Token)
============================================================
Status Code: 401
Response: {
  "error": "Token is missing",
  "message": "Authorization header with Bearer token is required"
}

[PASS] Unauthorized request correctly rejected!

============================================================
  TEST 4b: Unauthorized Request (Invalid Token)
============================================================
Status Code: 401
Response: {
  "error": "Invalid token",
  "message": "The provided token is not valid"
}

[PASS] Invalid token correctly rejected!

============================================================
  TEST 5: Rate Limiting
============================================================
Sending 50 concurrent requests to trigger rate limiting...
```

```
  Results:
    Successful requests:    12
    Rate-limited (429):     38

[PASS] Rate limiting is working! 38 requests were blocked.

[..] Waiting 5 seconds for worker to finish processing...

============================================================
  TEST 6: Database Log Verification
============================================================

--- Audit Logs (last 10) ---
+---------------------+-----------+--------------+----------------------------+----------+----------+
| Timestamp           | Service   | Request ID   | Action                     | Status   | Source   |
+=====================+===========+==============+============================+==========+==========+
| 2026-04-25 21:22:52 | worker    | 9d11617b...  | Task consumed from queue   | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:52 | worker    | fea82818...  | Task processed successfully | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:50 | worker    | fea82818...  | Task consumed from queue   | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:50 | worker    | 577ec48f...  | Task processed successfully | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:49 | api3      | 40928efa...  | Task queued to RabbitMQ    | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | 40928efa...  | JWT authentication         | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | 40928efa...  | Request received           | success  | client   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | dc4cc973...  | Task queued to RabbitMQ    | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | dc4cc973...  | JWT authentication         | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | dc4cc973...  | Request received           | success  | client   |
+---------------------+-----------+--------------+----------------------------+----------+----------+

--- Request States (last 15) ---
+---------------------+--------------+---------------+-----------+-------------------------------------+
| Timestamp           | Request ID   | State         | Service   | Details                             |
+=====================+==============+===============+===========+=====================================+
| 2026-04-25 21:22:52 | 9d11617b...  | CONSUMED      | worker    | Task consumed from RabbitMQ         |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:52 | fea82818...  | PROCESSED     | worker    | Task 'lb_test' processed successfully |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:50 | fea82818...  | CONSUMED      | worker    | Task consumed from RabbitMQ         |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:50 | 577ec48f...  | PROCESSED     | worker    | Task 'lb_test' processed successfully |
+---------------------+--------------+---------------+-----------+-------------------------------------+
```

```
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:49 | 40928efa...  | QUEUED        | api3      | Task published to RabbitMQ            |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | 40928efa...  | AUTHENTICATED | api3      | JWT validated for user 'admin'        |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | 40928efa...  | RECEIVED      | api3      | Request received from user 'admin'    |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | dc4cc973...  | QUEUED        | api3      | Task published to RabbitMQ            |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | dc4cc973...  | AUTHENTICATED | api3      | JWT validated for user 'admin'        |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | dc4cc973...  | RECEIVED      | api3      | Request received from user 'admin'    |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | 577ec48f...  | CONSUMED      | worker    | Task consumed from RabbitMQ           |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | 0df6e129...  | PROCESSED     | worker    | Task 'lb_test' processed successfully |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | a51a018b...  | QUEUED        | api2      | Task published to RabbitMQ            |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | 84f60cd4...  | QUEUED        | api1      | Task published to RabbitMQ            |
+---------------------+--------------+---------------+-----------+---------------------------------------+
| 2026-04-25 21:22:48 | 2a9df6ee...  | QUEUED        | api2      | Task published to RabbitMQ            |
+---------------------+--------------+---------------+-----------+---------------------------------------+

  Total audit logs:    1692
  Total state records: 1692

--- Full State Chain (Latest Request) ---
  Request ID: fff496ad-cb00-41f8-be1e-c59bbf23a4dd
    -> RECEIVED        | api1     | 2026-04-16 21:14:54
    -> AUTHENTICATED   | api1     | 2026-04-16 21:14:54
    -> QUEUED          | api1     | 2026-04-16 21:14:54
    -> CONSUMED        | worker   | 2026-04-16 21:15:41
    -> PROCESSED       | worker   | 2026-04-16 21:15:43

[PASS] Database logs verified!

============================================================
  ALL TESTS COMPLETED
============================================================

[PASS] Review the results above to verify system functionality.
```

8.2 RabbitMQ Management UI

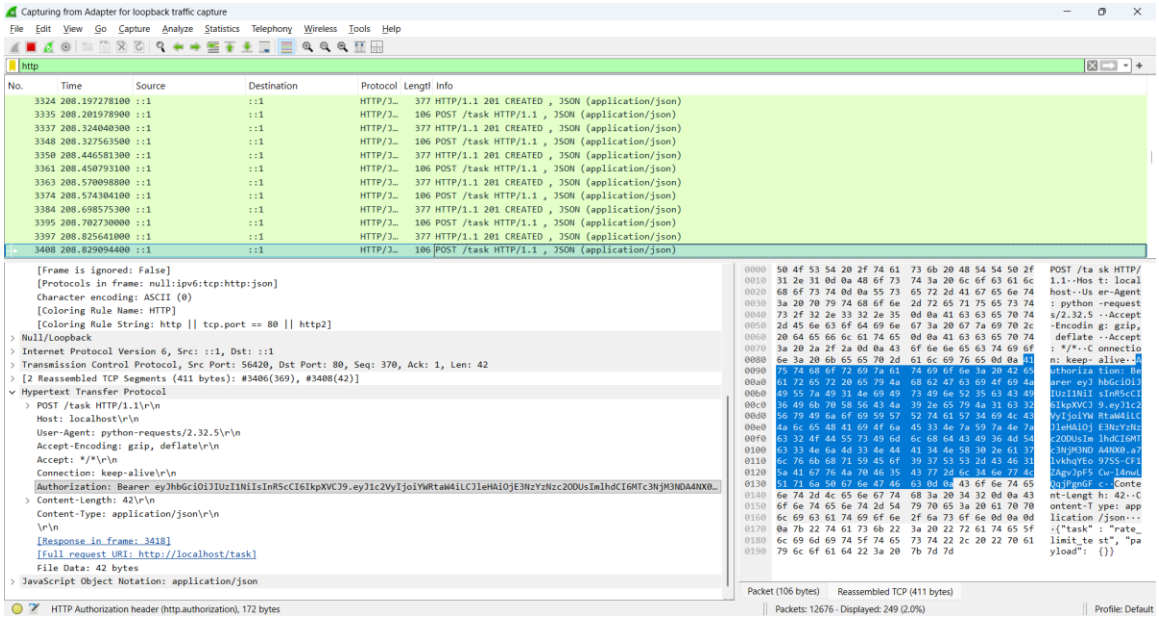


8.3 Database Audit Logs & Request States

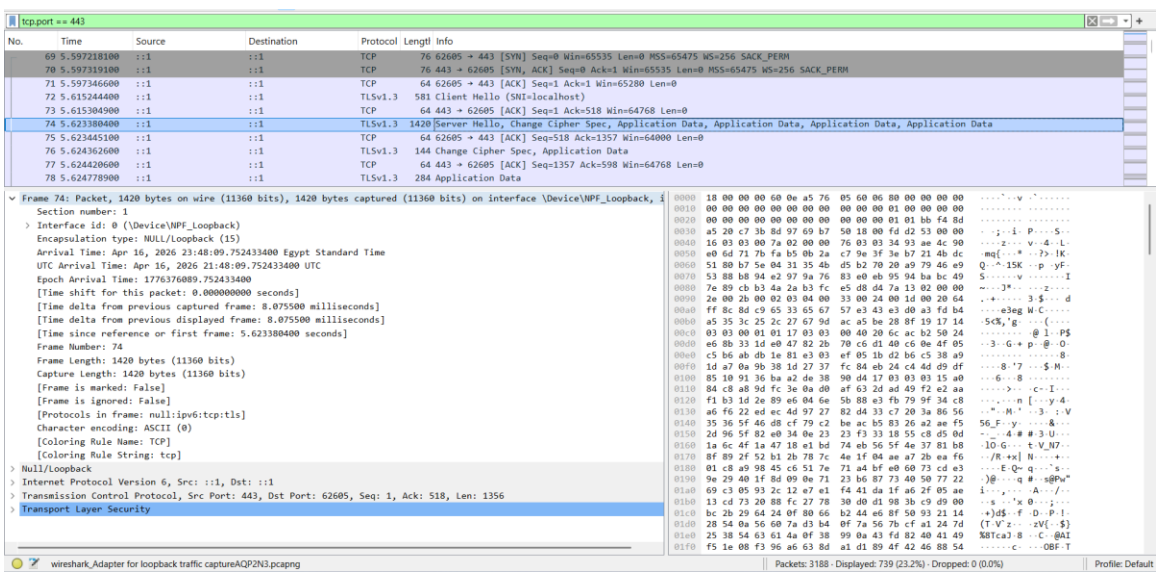
```
============================================================
  TEST 5: Rate Limiting
============================================================
Sending 50 concurrent requests to trigger rate limiting...

  Results:
    Successful requests:    12
    Rate-limited (429):     38

[PASS] Rate limiting is working! 38 requests were blocked.

[..] Waiting 5 seconds for worker to finish processing...

============================================================
  TEST 6: Database Log Verification
============================================================

--- Audit Logs (last 10) ---
+---------------------+-----------+--------------+----------------------------+----------+----------+
| Timestamp           | Service   | Request ID   | Action                     | Status   | Source   |
+=====================+===========+==============+============================+==========+==========+
| 2026-04-25 21:22:52 | worker    | 9d11617b...  | Task consumed from queue   | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:52 | worker    | fea82818...  | Task processed successfully | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:50 | worker    | fea82818...  | Task consumed from queue   | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:50 | worker    | 577ec48f...  | Task processed successfully | success  | worker   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:49 | api3      | 40928efa...  | Task queued to RabbitMQ    | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | 40928efa...  | JWT authentication         | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | 40928efa...  | Request received           | success  | client   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | dc4cc973...  | Task queued to RabbitMQ    | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | dc4cc973...  | JWT authentication         | success  | api3     |
+---------------------+-----------+--------------+----------------------------+----------+----------+
| 2026-04-25 21:22:48 | api3      | dc4cc973...  | Request received           | success  | client   |
+---------------------+-----------+--------------+----------------------------+----------+----------+
```

```
--- Request States (last 15) ---
+---------------------+--------------+---------------+-----------+-------------------------------------+
| Timestamp           | Request ID   | State         | Service   | Details                             |
+=====================+==============+===============+===========+=====================================+
| 2026-04-25 21:22:52 | 9d11617b...  | CONSUMED      | worker    | Task consumed from RabbitMQ         |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:52 | fea82818...  | PROCESSED     | worker    | Task 'lb_test' processed successfully |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:50 | fea82818...  | CONSUMED      | worker    | Task consumed from RabbitMQ         |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:50 | 577ec48f...  | PROCESSED     | worker    | Task 'lb_test' processed successfully |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:49 | 40928efa...  | QUEUED        | api3      | Task published to RabbitMQ          |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | 40928efa...  | AUTHENTICATED | api3      | JWT validated for user 'admin'      |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | 40928efa...  | RECEIVED      | api3      | Request received from user 'admin'  |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | dc4cc973...  | QUEUED        | api3      | Task published to RabbitMQ          |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | dc4cc973...  | AUTHENTICATED | api3      | JWT validated for user 'admin'      |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | dc4cc973...  | RECEIVED      | api3      | Request received from user 'admin'  |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | 577ec48f...  | CONSUMED      | worker    | Task consumed from RabbitMQ         |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | 0df6e129...  | PROCESSED     | worker    | Task 'lb_test' processed successfully |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | a51a018b...  | QUEUED        | api2      | Task published to RabbitMQ          |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | 84f60cd4...  | QUEUED        | api1      | Task published to RabbitMQ          |
+---------------------+--------------+---------------+-----------+-------------------------------------+
| 2026-04-25 21:22:48 | 2a9df6ee...  | QUEUED        | api2      | Task published to RabbitMQ          |
+---------------------+--------------+---------------+-----------+-------------------------------------+

  Total audit logs:    1692
  Total state records: 1692

--- Full State Chain (Latest Request) ---
  Request ID: fff496ad-cb00-41f8-be1e-c59bbf23a4dd
    -> RECEIVED        | api1     | 2026-04-16 21:14:54
    -> AUTHENTICATED   | api1     | 2026-04-16 21:14:54
    -> QUEUED          | api1     | 2026-04-16 21:14:54
    -> CONSUMED        | worker   | 2026-04-16 21:15:41
    -> PROCESSED       | worker   | 2026-04-16 21:15:43

[PASS] Database logs verified!
```

8.4 Wireshark — HTTP Mode (Traffic Exposed)



8.5 Wireshark — HTTPS Mode (Traffic Encrypted)



---

9. Conclusion

This project successfully implements a secure distributed system that meets all specified requirements:

1. **Load Balancing**: Nginx distributes requests evenly across three API instances using round-robin.

2. **HTTPS Communication**: All client-server traffic is encrypted with TLS 1.2/1.3.
3. **Authentication**: JWT-based authentication protects all task endpoints.
4. **Asynchronous Processing**: RabbitMQ enables decoupled, reliable task processing.
5. **Rate Limiting**: Nginx prevents abuse by limiting requests per client IP.
6. **Audit Logging**: Every event is recorded in PostgreSQL with full details.
7. **State Tracking**: Each request's lifecycle is tracked from RECEIVED to PROCESSED.
8. **Security Analysis**: Wireshark captures prove that HTTP exposes sensitive data while HTTPS encrypts it completely.

The MITM simulation clearly demonstrates why HTTPS is essential in any production distributed system. Without encryption, authentication tokens and sensitive data are trivially interceptable. With HTTPS, the same data becomes completely unreadable to any network-level attacker.

---

10. GitHub Repository

Repository URL: https://github.com/nadine-rasmy23/Secure-Distributed-System-with-Observability-and-Security-Analysis